

HW4_加分

主題目的 code

```
1 # 1. Remove genes with name starting AFFX...(控制探針,共 49 個)
2 MLL_train <- read.delim("MLL_train.txt")
3 data <- MLL_train[-c(1:49), ]
4
5 # 正規化
6 min_max_norm <- function(x){
7   (x - min(x)) / (max(x) - min(x))
8 }
9
10 # 對每一 row 計算 t-test
11 count_t <- function(group_a, group_b){
12   t_value <- numeric(nrow(group_a))
13   for(i in 1:nrow(group_a)){
14     t_value[[i]] <- t.test(group_a[i, ], group_b[i, ])$statistic
15   }
16   return(t_value)
17 }
18
19 # 2. Normalize data by each row(0~1)
20 data_norm <- t(apply(data[, 3:ncol(data)], 1, min_max_norm))
21
22 col_name <- colnames(data_norm)
23 labels <- sub("_.*", "", col_name)
24 table(labels)
25
26 # 3. Find top 10 differentially expressed genes using t-test, 每組一次(該組 vs 其他),
27 # 條件:mean(該組) > mean(其他)
28 # * 三組各 10 個 → 合併成 30 selected genes
29 # ALL
30 ALL_data <- data_norm[, which(labels == "ALL")] # 只有 ALL 的欄位資料
31 ALL_others_data <- data_norm[, -which(labels == "ALL")] # 除了 ALL 的欄位資料
32
33 ALL_mean <- apply(ALL_data, 1, mean) # 計算只有 ALL 欄位的每一列平均值
34 ALL_others_data_mean <- apply(ALL_others_data, 1, mean) # 計算除了 ALL 欄位的每一列平均值
35
36 # 只保留 mean(該組) > mean(其他)
37 ALL_data_filter <- ALL_data[which(ALL_mean > ALL_others_data_mean), ]
38 ALL_others_data_filter <- ALL_others_data[which(ALL_mean > ALL_others_data_mean), ]
39
40 # 計算t-test
41 ALL_t <- count_t(ALL_data_filter, ALL_others_data_filter)
42
43 # 由大到小排列
44 ALL_t_sort <- order(ALL_t, decreasing = TRUE)
45
46 # 透過 order 回傳的前十個索引找到在 ALL_data_filter 的 gene
47 ALL_top_10_genes <- ALL_data_filter[ALL_t_sort[1:10], ]
48 ALL_top_10_genes
```

```

50 ## 以下同理
51
52 # MLL
53 MLL_data <- data_norm[, which(labels == "MLL")]
54 MLL_others_data <- data_norm[, -which(labels == "MLL")]
55
56 MLL_mean <- apply(MLL_data, 1, mean) # 計算只有 MLL 欄位的每一列平均值
57 MLL_others_data_mean <- apply(MLL_others_data, 1, mean) # 計算除了 MLL 欄位的每一列平均值
58
59 # 只保留 mean(該組) > mean(其他)
60 MLL_data_filter <- MLL_data[which(MLL_mean > MLL_others_data_mean), ]
61 MLL_others_data_filter <- MLL_others_data[which(MLL_mean > MLL_others_data_mean), ]
62
63 MLL_t <- count_t(MLL_data_filter, MLL_others_data_filter)
64 MLL_t_sort <- order(MLL_t, decreasing = TRUE)
65
66 MLL_top_10_genes <- MLL_data_filter[MLL_t_sort[1:10], ]
67 MLL_top_10_genes
68
69 # AML
70 AML_data <- data_norm[, which(labels == "AML")]
71 AML_others_data <- data_norm[, -which(labels == "AML")]
72
73 AML_mean <- apply(AML_data, 1, mean) # 計算只有 MLL 欄位的每一列平均值
74 AML_others_data_mean <- apply(AML_others_data, 1, mean) # 計算除了 MLL 欄位的每一列平均值
75
76 # 只保留 mean(該組) > mean(其他)
77 AML_data_filter <- AML_data[which(AML_mean > AML_others_data_mean), ]
78 AML_others_data_filter <- AML_others_data[which(AML_mean > AML_others_data_mean), ]
79
80 AML_t <- count_t(AML_data_filter, AML_others_data_filter)
81 AML_t_sort <- order(AML_t, decreasing = TRUE)
82
83 AML_top_10_genes <- AML_data_filter[AML_t_sort[1:10], ]
84 AML_top_10_genes
85
86 # 將三組各 10 個 → 合併成 30 selected genes
87 selected_genes <- c(rownames(ALL_top_10_genes),
88                   rownames(MLL_top_10_genes),
89                   rownames(AML_top_10_genes))
90
91 BiocManager::install("e1071")
92 library("e1071")

```

☆☆☆ 加分題 A(最難)

三輪跑完後請回報：

1. 每個 fold 中 ALL / MLL / AML 各有幾個樣本

2. 每一個 fold 的 confusion matrix

3. 整體(overall)的 confusion matrix

✎ 這裡的 confusion matrix 同樣是三類別(3-class)的 3×3 表

(ALL / MLL / AML)

fold 存的是主題目篩選出的 30 個基因

```
fold <- data_norm[selected_genes, ]
```

依樣本的類別標籤 (ALL / MLL / AML) 分層把樣本切成 3 folds, 讓每個 fold 都包含三個類別的樣本,

且盡可能維持原本的類別比例。 ALL 7/7/6、MLL 6/6/5、AML 7/7/6

```
ALL_fold1 <- fold[, 1:7]
ALL_fold2 <- fold[, 8:14]
ALL_fold3 <- fold[, 15:20]
```

```
MLL_fold1 <- fold[, 21:26]
MLL_fold2 <- fold[, 27:32]
MLL_fold3 <- fold[, 33:37]
```

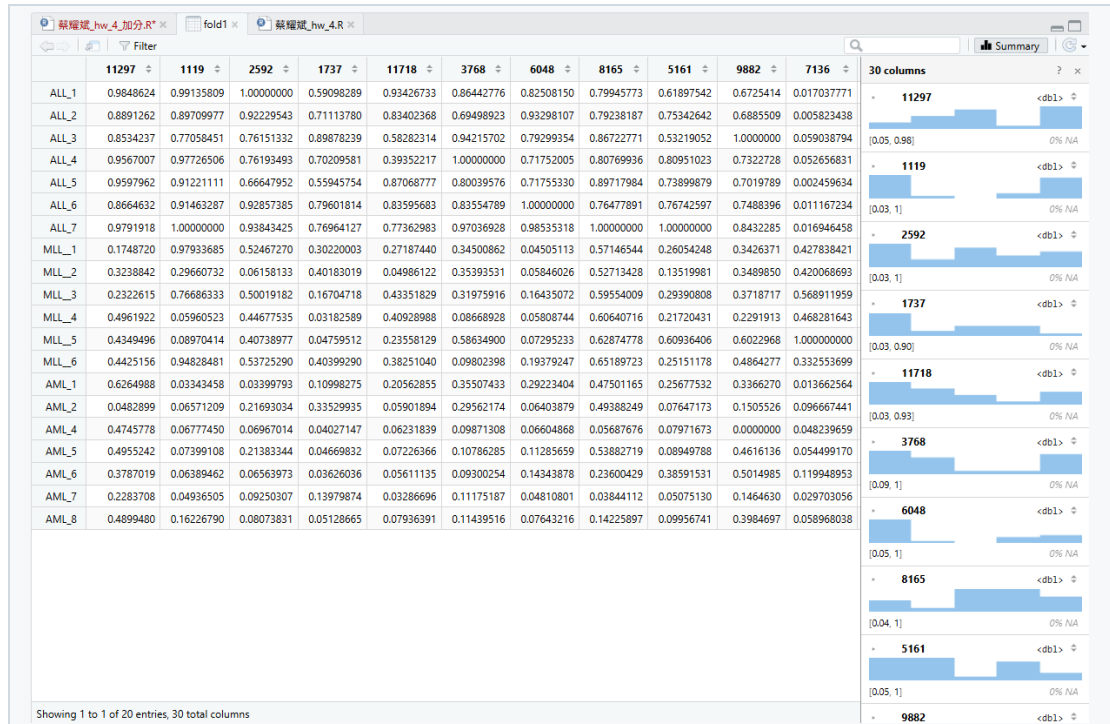
```
AML_fold1 <- fold[, 38:44]
AML_fold2 <- fold[, 45:51]
AML_fold3 <- fold[, 52:57]
```

```
fold1 <- rbind(t(ALL_fold1), t(MLL_fold1), t(AML_fold1))
fold2 <- rbind(t(ALL_fold2), t(MLL_fold2), t(AML_fold2))
fold3 <- rbind(t(ALL_fold3), t(MLL_fold3), t(AML_fold3))
```

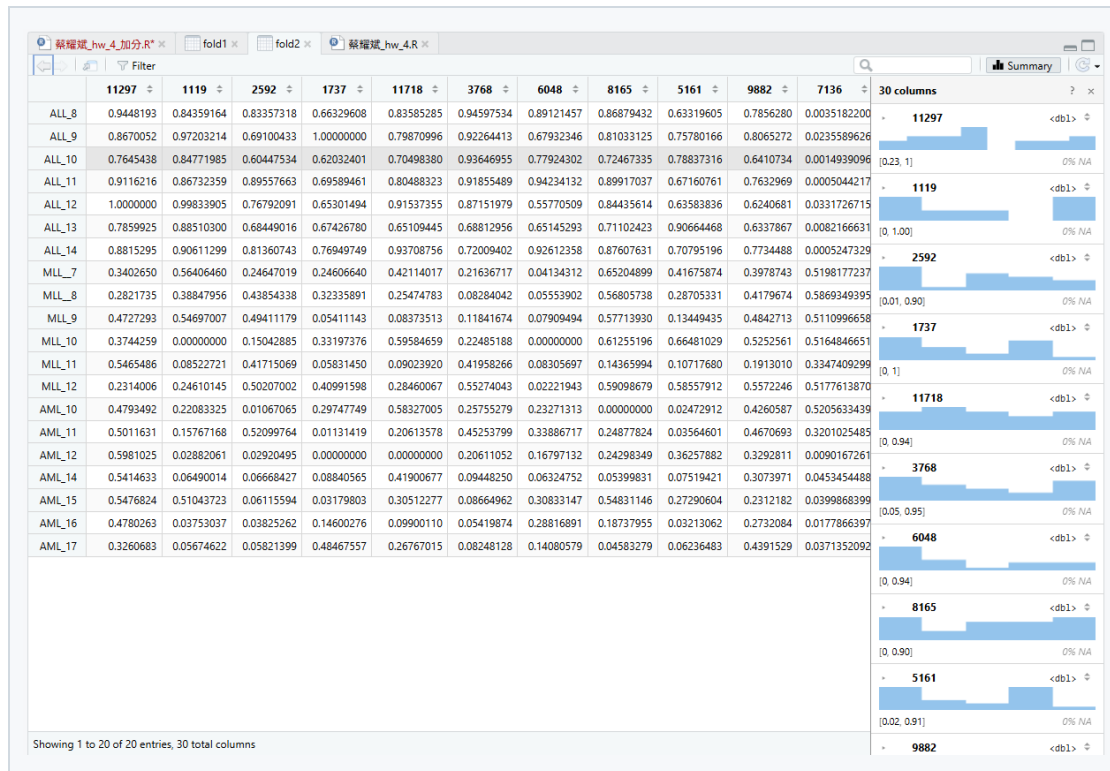
```
folds <- list(fold1, fold2, fold3)
```

```
fold_labels <- list(sub("_.*", "", row.names(fold1)),
                    sub("_.*", "", row.names(fold2)),
                    sub("_.*", "", row.names(fold3)))
```

fold1 觀察



fold2 觀察



fold3 觀察

蘇耀斌_hw_4_加分.R* × fold1 × fold2 × fold3 × 蘇耀斌_hw_4.R ×

Filter

Summary

	11297	1119	2592	1737	11718	3768	6048	8165	5161	9882	7136	30 columns
ALL_15	0.91899332	0.893900848	0.69454105	0.81952946	0.87071007	0.88950797	0.4820084565	0.794466382	0.87817603	0.6945669	0.0468279	11297 <db1>
ALL_16	0.92280189	0.947681733	0.69868969	0.59091567	0.90824907	0.74706771	0.5627890503	0.773927226	0.93210808	0.7149539	0.0347679	[0, 0.97] 0% NA
ALL_17	0.76031473	0.833965651	0.68762608	0.44321842	0.75849299	0.32125270	0.0502518763	0.770783988	0.37326767	0.7660133	0.0319181	1119 <db1>
ALL_18	0.97428097	0.943125942	0.86308082	0.37990672	0.53104096	0.38855687	0.6425075417	0.767677852	0.66857875	0.4404668	0.0437666	[0.000706, 0.95] 0% NA
ALL_19	0.91162640	0.811428936	0.62923002	0.82093654	0.46962864	0.83188232	0.6915220938	0.786479257	0.82580609	0.7090224	0.0087737	2592 <db1>
ALL_20	0.66823692	0.880863536	0.64966630	0.46379829	1.00000000	0.51358988	0.6087870240	0.902696134	0.42856748	0.6279479	0.0000000	[0, 0.86] 0% NA
MLL_13	0.27503849	0.644939188	0.56943586	0.04344806	0.45216760	0.36908668	0.0345411431	0.609767013	0.10997803	0.4141065	0.4547034	1737 <db1>
MLL_14	0.52309669	0.145581416	0.68211249	0.15712961	0.43086278	0.54699578	0.0576468461	0.672948695	0.56377818	0.7496976	0.5894780	[0, 0.02, 0.62] 0% NA
MLL_15	0.02010364	0.036287796	0.25507965	0.16755622	0.22452904	0.54346115	0.0353637596	0.549288236	0.08264537	0.4351694	0.4582842	11718 <db1>
MLL_16	0.00000000	0.589350438	0.38542424	0.26332996	0.26076399	0.24859907	0.0149115089	0.467009800	0.13447922	0.3835109	0.5308939	[0, 0.06, 1] 0% NA
MLL_17	0.37488664	0.000706416	0.00000000	0.11896992	0.58457051	0.00000000	0.0006884278	0.369537258	0.46007472	0.3322054	0.3218738	3768 <db1>
AML_18	0.39769749	0.202348855	0.05102491	0.30766556	0.31037628	0.19295349	0.5426748315	0.528580930	0.33866480	0.2963614	0.0301668	[0, 0.02, 0.62] 0% NA
AML_19	0.28962360	0.017109220	0.01703919	0.22245302	0.26465788	0.20288502	0.4097872155	0.006139406	0.00000000	0.2002602	0.3073069	11718 <db1>
AML_20	0.47699337	0.052490055	0.05379270	0.02447004	0.32511940	0.07621692	0.0511534427	0.426416648	0.05566819	0.3452830	0.1333562	[0, 0.06, 1] 0% NA
AML_21	0.52225889	0.072722035	0.07480963	0.13391233	0.07023344	0.10599503	0.0708702330	0.231517120	0.08750117	0.2928826	0.2070014	3768 <db1>
AML_22	0.11658468	0.031634333	0.03212784	0.25901877	0.05916018	0.25862149	0.0308287935	0.334868831	0.02285381	0.3372009	0.0118498	[0, 0.89] 0% NA
AML_23	0.45132705	0.024306657	0.02451587	0.06091078	0.32164797	0.03473564	0.2370836203	0.540204547	0.50187616	0.4352288	0.3100874	6048 <db1>
												[0.000668, 0.69] 0% NA
												8165 <db1>
												[0.00614, 0.90] 0% NA
												5161 <db1>
												[0, 0.93] 0% NA
												9882 <db1>

Showing 1 to 17 of 17 entries, 30 total columns

```
# 對每一輪 cross-validation:  
  
# • 用兩個 fold 當 training set  
  
# • 用剩下一個 fold 當 testing set  
  
# • 只用 training 樣本訓練 SVM  
  
# • 預測 testing fold 的類別標籤  
  
# • 重複直到每個 fold 都當過一次 testing set
```

```
# AI  
all_pred <- character(0)  
all_true <- character(0)  
  
for (i in 1:3){  
  # 先選一個 fold 當 test set  
  x_test <- folds[[i]]  
  y_test <- folds_labels[[i]]  
  # 用剩下兩個 fold 當 training set  
  x_train <- rbind(folds[-i][[1]], folds[-i][[2]])  
  y_train <- c(folds_labels[-i][[1]],  
              folds_labels[-i][[2]])  
  y_train <- factor(y_train)  
  
  svm_model <- svm(x_train, y_train)  
  pred <- predict(svm_model, x_test)  
  
  cat("==== Fold", i, "====\n")  
  print(table(y_test))  
  cat("\n")  
  print(table(Predicted = pred, Actual = y_test))  
  cat("\n")  
  all_pred <- c(all_pred, as.character(pred))  
  all_true <- c(all_true, as.character(y_test))  
}
```

```
cat("==== Overall ====\n")
print(table(Predicted = all_pred, Actual = all_true))
```

Output:

```
==== Fold 1 ====
y_test
ALL AML MLL
  7  7  6
      Actual
Predicted ALL AML MLL
      ALL  7  0  0
      AML  0  7  0
      MLL  0  0  6

==== Fold 2 ====
y_test
ALL AML MLL
  7  7  6
      Actual
Predicted ALL AML MLL
      ALL  7  0  0
      AML  0  7  0
      MLL  0  0  6

==== Fold 3 ====
y_test
ALL AML MLL
  6  6  5
      Actual
Predicted ALL AML MLL
      ALL  6  0  0
      AML  0  6  1
      MLL  0  0  4
```

```
> cat("==== Overall ====\n")
==== Overall ====
> print(table(Predicted = all_pred, Actual = all_true))
```

	Actual		
Predicted	ALL	AML	MLL
ALL	20	0	0
AML	0	20	1
MLL	0	0	16

AI

```
#1 Remove genes with name starting AFFX_(控制基因共 49 個) MLL_train <- read.delim("MLL_train.txt") data <- MLL_train[-c(1:49),] # 正歸化 min_max...
```

PASTED

請幫看看程式碼有沒有問題

整體結構已經很好，迴圈那段（最後的 for）是正確的。但有幾個地方要修，其中一個會直接影響主題目的正確性。我按嚴重程度排。

要修的問題

問題 1（影響主題目）：`selected_genes` 少了 `unique()`。

你之前的版本有寫 `selected_genes <- unique(...)`，這一版拿掉了。三組的前 10 名可能有重複基因，不去重的話，`fold <- data_norm[selected_genes, ]` 會出現重複的列。雖然不一定報錯，但基因會被重複計入，不符合題目「合併成 30 selected genes」的原意。補回來：

```
r
selected_genes <- c(rownames(ALL_top_10_genes),
                    rownames(MLL_top_10_genes),
                    rownames(AML_top_10_genes))
selected_genes <- unique(selected_genes)
```

Copy

問題 2（無用且會誤導）：迴圈前面那兩行 `x_train` / `y_train` 要刪掉。

```
r
x_train <- rbind(folds[-1][[1]], folds[-1][[2]])
y_train <- rbind(factor(folds_labels[-1][[1]]), folds_labels[-1][[2]])
```

這兩行寫在迴圈外面，而且第二行還用了 `rbind`（會把 factor 疊成 2 列矩陣，是錯的）。它們完全沒被用到——迴圈裡面已經重新算過正確的 `x_train` / `y_train` 了。這兩行是殘留的草稿，留著只會誤導，直接刪掉。

加分題還缺兩項回報

加分題要求回報三件事，你目前迴圈只印了「每個 fold 的 confusion matrix」，還差另外兩項：

(1) 每個 fold 各類別的樣本數，在迴圈裡加一行：

```
r
cat("=== Fold", i, "===\n")
print(table(y_test))          # 這個 fold 各類別數量
```

(3) 整體 confusion matrix，需要在迴圈外累加。完整版長這樣：

```
r
all_pred <- character(0)
all_true <- character(0)

for (i in 1:3){
  x_test <- folds[[i]]
  y_test <- folds_labels[[i]]

  x_train <- rbind(folds[-i][[1]], folds[-i][[2]])
  y_train <- c(folds_labels[-i][[1]], folds_labels[-i][[2]])
  y_train <- factor(y_train)

  svm_model <- svm(x_train, y_train)
  pred <- predict(svm_model, x_test)

  cat("=== Fold", i, "===\n")
  print(table(y_test))          # (1) 各類別樣本數
  print(table(Predicted = pred, Actual = y_test))  # (2) 該fold混淆矩陣

  all_pred <- c(all_pred, as.character(pred))
  all_true <- c(all_true, as.character(y_test))
}

# (3) 整體 confusion matrix
cat("=== Overall ===\n")
print(table(Predicted = all_pred, Actual = all_true))
```

★ 加分題 B

改用 SRBCT_train.txt 這份資料(四類 : EWS 23 / RMS 20 / NB 12 / BL 8), 把主題目的 t-test 換成 Wilcoxon rank-sum test

(wilcox.test, 無母數檢定)來挑選差異表現基因，每類各取 top 10。

讀檔

```
data_SRBCT <- read.delim("SRBCT_train.txt")
data2 <- as.matrix(data_SRBCT[, -c(1:2)])
Image_id <- data_SRBCT[,1]

# 正規化
min_max_norm <- function(x){
  (x - min(x)) / (max(x) - min(x))
}
data2_norm <- t(apply(data2, 1, min_max_norm))

# 取得所有欄位名稱
cols2 <- colnames(data2)
labels2 <- sub("\\..*", "", cols2)
table(labels) # 觀察
labels2       # 觀察
```

Output:

[illegible]

套運提示的公式，每類各取 top 10

```
# 套運提示的公式，每類各取 top 10
EWS_p <- apply(data2_norm, 1, function(x)
  wilcox.test(x[which(labels2 == "EWS")], x[-which(labels2 == "EWS")], alternative = "greater")$p.value)
EWS_p_top10 <- order(EWS_p)[1:10]

BL_p <- apply(data2_norm, 1, function(x)
  wilcox.test(x[which(labels2 == "BL")], x[-which(labels2 == "BL")], alternative = "greater")$p.value)
BL_p_top10 <- order(BL_p)[1:10]

NB_p <- apply(data2_norm, 1, function(x)
  wilcox.test(x[which(labels2 == "NB")], x[-which(labels2 == "NB")], alternative = "greater")$p.value)
NB_p_top10 <- order(NB_p)[1:10]

RMS_p <- apply(data2_norm, 1, function(x)
  wilcox.test(x[which(labels2 == "RMS")], x[-which(labels2 == "RMS")], alternative = "greater")$p.value)
RMS_p_top10 <- order(RMS_p)[1:10]

EWS_top10_id <- Image_id[EWS_p_top10]
BL_top10_id <- Image_id[BL_p_top10]
NB_top10_id <- Image_id[NB_p_top10]
RMS_top10_id <- Image_id[RMS_p_top10]

EWS_top10_id
BL_top10_id
NB_top10_id
RMS_top10_id
```

Output:

```
> EWS_top10_id
[1] 1435862 814260 377461 770394 357031 43733 866702 491565 52076 1473131
> BL_top10_id
[1] 236282 241412 183337 47475 745019 624360 1469292 814526 769657 80109
> NB_top10_id
[1] 812105 325182 308231 134748 81518 756401 786084 629896 244637 377048
> RMS_top10_id
[1] 796258 784224 244618 207274 769716 296448 143306 789253 1409509 813841
```

★ 加分題 c

沿用主題目在 **training set** 上選出的 30 個基因與訓練好的 **SVM** 模型，套用到 **MLL_test.txt** 上做預測，並做出 **test set** 的 **confusion matrix**(同樣是三類別的 3×3 表)。

MLL_test.txt 共 15 個樣本(**ALL 4 / MLL 3 / AML 8**)，基因列與 **MLL_train.txt** 完全對齊(同樣 12582 列、同樣順序)，所以選出的基因可以直接用相同的列索引取出。

重點:基因的挑選與模型的訓練都只能用 **training set**; **test set** 只在最後拿來評估,不可以參與選基因或訓練。

主題目篩選 30 genes & svm model (加分題A以寫過，再拿下來用一次)

```
selected_genes <- c(rownames(ALL_top_10_genes),
                    rownames(MLL_top_10_genes),
                    rownames(AML_top_10_genes))

# 讀檔
MLL_test <- read.delim("MLL_test.txt")
test_data <- MLL_test[-c(1:49), ]

# 正規化 (function 在加分題A，所以直接呼叫)
test_data_norm <- t(apply(test_data[, 3:ncol(test_data)], 1,
min_max_norm))

# 從 test_data 提出 train_data 選出的 30 個基因列
test_data_30 <- test_data_norm[selected_genes, ]

# 清洗欄位名稱
test_data_col_name <- colnames(test_data_30)
test_data_labels <- sub("_.*", "", test_data_col_name)
test_data_labels      # 觀查
table(test_data_labels) # 觀查
```

Output:

```
> test_data_labels # 觀察
[1] "ALL" "ALL" "ALL" "ALL" "MLL" "MLL" "MLL" "MLL" "AML" "AML" "AML" "AML" "AML" "AML" "AML" "AML"
> table(test_data_labels) # 觀察
test_data_labels
ALL AML MLL
  4   8   3
```

將 test_data 喂進去 svm model 跑結果

```
## 主題目在 training set 上選出的 30 個基因與訓練好的 SVM 模型
## data_norm、selected_genes、labels 在加分題A已執行
train_data <- data_norm[selected_genes, ]
x_train <- t(train_data)
y_train <- factor(labels)
svm_model <- svm(x_train, y_train)

x_test2 <- t(test_data_30)
y_test2 <- factor(test_data_labels)

pred2 <- predict(svm_model, x_test2)

confusion_matrix <- table(Predicted = pred2,
                          Actual = y_test2)
confusion_matrix
```

Output:

```
> confusion_matrix
      Actual
Predicted ALL AML MLL
    ALL    4  0  0
    AML    0  8  0
    MLL    0  0  3
```